

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<!-- This site is hosted on Netlify. Anyone can build and deploy a site
like this one for free: https://netlify.new/?utm_campaign=ai-legible&utm_source=comment&utm_medium=referral&utm_id=0f46a784-
5361-4cd0-a4b2-df1aa15a7c87
Netlify hosting facts for this site: static/SSR served via Netlify Edge. -->
<meta name="hosting-provider" content="Netlify">
<meta name="netlify-deploy" content="https://netlify.new/?utm_campaign=ai-
legible&utm_source=meta&utm_medium=referral&utm_id=0f46a784-5361-4cd0-a4b2-df1aa15a7c87">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>The Dragon Egg Cave — Entropy & Cross-Entropy</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link href="https://fonts.googleapis.com/css2?
family=Baloo+2:wght@500;700;800&family=Inter:wght@400;500;600;700&family=JetBrains+Mono:wght@500&display=swap"
rel="stylesheet">
<style>
:root{
--cave-black:#140a22;
--cave-deep:#1d1035;
--cave-panel:#261541;
--cave-panel-2:#2f1b4f;
--ember:#e8541e;
--ember-bright:#ff7a3d;
--gold:#f2b705;
--crystal:#37d6c4;
--parchment:#f6efe1;
--ink:#241636;
--danger:#ff4d4d;
--line:rgba(246,239,225,0.14);
--text-hi:#f6efe1;
--text-mid:#cabfe0;
--text-dim:#8f81ad;
--radius-lg:22px;
--radius-md:14px;
}
*{box-sizing:border-box;}
html{scroll-behavior:smooth;}
body{
margin:0;
background:var(--cave-black);
color:var(--text-hi);
font-family:'Inter',system-ui,-apple-system,sans-serif;
line-height:1.55;
-webkit-font-smoothing:antialiased;
}
h1,h2,h3,.display{
font-family:'Baloo 2','Inter',sans-serif;
font-weight:800;
letter-spacing:0.2px;
}
.mono{font-family:'JetBrains Mono',monospace;}
img,svg{max-width:100%;}
.wrap{max-width:1040px;margin:0 auto;padding:0 24px;}
.prose{max-width:660px;}
```

```

.prose p{color:var(--text-mid);font-size:1.05rem;}

/* ----- nav ----- */
nav.topnav{
position:sticky;top:0;z-index:50;
background:rgba(20,10,34,0.86);
backdrop-filter:blur(10px);
border-bottom:1px solid var(--line);
}
.topnav-inner{
max-width:1040px;margin:0 auto;padding:12px 24px;
display:flex;align-items:center;gap:18px;overflow-x:auto;
}
.topnav-brand{font-family:'Baloo 2';font-weight:700;font-size:1.02rem;color:var(--gold);white-space:nowrap;}
.topnav a{
color:var(--text-mid);text-decoration:none;font-size:0.88rem;font-weight:600;
white-space:nowrap;padding:6px 4px;border-bottom:2px solid transparent;
}
.topnav a:hover{color:var(--text-hi);border-bottom-color:var(--ember);}

/* ----- hero ----- */
header.hero{
position:relative;
padding:64px 24px 56px;
background:
radial-gradient(ellipse 900px 500px at 50% -10%, rgba(232,84,30,0.35), transparent 60%),
radial-gradient(ellipse 700px 400px at 80% 110%, rgba(55,214,196,0.14), transparent 60%),
var(--cave-deep);
border-bottom:1px solid var(--line);
overflow:hidden;
}
.eyebrow-tag{
display:inline-block;
font-family:'JetBrains Mono',monospace;
font-size:0.78rem;
color:var(--gold);
background:rgba(242,183,5,0.1);
border:1px solid rgba(242,183,5,0.35);
padding:6px 12px;
border-radius:8px;
margin-bottom:22px;
}
.hero-title{font-size:clamp(2.3rem,5.4vw,3.6rem);line-height:1.05;margin:0 0 18px;color:var(--parchment);}
.hero-title em{font-style:normal;color:var(--ember-bright);}
.hero-sub{font-size:1.15rem;color:var(--text-mid);max-width:600px;margin:0 0 30px;}
.hero-grid{display:grid;grid-template-columns:1.1fr 0.9fr;gap:40px;align-items:center;}
@media (max-width:760px){.hero-grid{grid-template-columns:1fr;}}
.badge-row{display:flex;gap:10px;flex-wrap:wrap;margin-top:10px;}
.badge{
font-size:0.82rem;font-weight:600;color:var(--text-hi);
background:var(--cave-panel);border:1px solid var(--line);
padding:7px 12px;border-radius:100px;
}

/* ----- cave scene svg ----- */

```

```
.cave-scene{width:100%;display:block;filter:drop-shadow(0 20px 40px rgba(0,0,0,0.5));}
```

```
/* ----- sections ----- */
```

```
section{padding:72px 24px;border-bottom:1px solid var(--line);}
```

```
section.alt{background:var(--cave-deep);}
```

```
.section-kicker{
```

```
font-family:'JetBrains Mono',monospace;color:var(--crystal);font-size:0.82rem;
```

```
margin:0 0 10px;
```

```
}
```

```
.section-title{font-size:clamp(1.6rem,3.4vw,2.3rem);margin:0 0 20px;color:var(--parchment);}
```

```
/* ----- story card ----- */
```

```
.story-card{
```

```
background:var(--cave-panel);
```

```
border:1px solid var(--line);
```

```
border-radius:var(--radius-lg);
```

```
padding:30px 32px;
```

```
margin-top:26px;
```

```
}
```

```
.story-card p{color:var(--text-mid);font-size:1.06rem;margin:0 0 14px;}
```

```
.story-card p:last-child{margin-bottom:0;}
```

```
/* ----- interactive panel ----- */
```

```
.panel{
```

```
background:var(--cave-panel);
```

```
border:1px solid var(--line);
```

```
border-radius:var(--radius-lg);
```

```
padding:32px;
```

```
margin-top:28px;
```

```
}
```

```
.panel-grid{display:grid;grid-template-columns:1fr 1fr;gap:32px;align-items:start;}
```

```
@media (max-width:800px){.panel-grid{grid-template-columns:1fr;}}
```

```
.egg-ring{
```

```
position:relative;width:100%;aspect-ratio:1/1;max-width:340px;margin:0 auto;
```

```
}
```

```
.egg{
```

```
position:absolute;width:56px;height:70px;transform:translate(-50%,-50%);
```

```
display:flex;align-items:center;justify-content:center;
```

```
font-size:2rem;cursor:pointer;
```

```
filter:drop-shadow(0 6px 10px rgba(0,0,0,0.4));
```

```
transition:transform .15s ease;
```

```
user-select:none;
```

```
}
```

```
.egg:hover{transform:translate(-50%,-50%) scale(1.08);}
```

```
.egg.selected{transform:translate(-50%,-50%) scale(1.16);}
```

```
.egg.selected .egg-shape{outline:3px solid var(--gold);border-radius:50%;}
```

```
.egg.true-egg{transform:translate(-50%,-50%) scale(1.2);}
```

```
.egg-pct{
```

```
position:absolute;bottom:-20px;left:50%;transform:translateX(-50%);
```

```
font-size:0.68rem;color:var(--text-dim);font-family:'JetBrains Mono',monospace;white-space:nowrap;
```

```
}
```

```
.egg-center-icon{
```

```
position:absolute;top:50%;left:50%;transform:translate(-50%,-50%);
```

```
font-size:1.9rem;opacity:0.5;
```

```

}

label.field-label{display:block;font-size:0.86rem;color:var(--text-mid);font-weight:600;margin-bottom:8px;}
input[type=range]{
width:100%;accent-color:var(--ember);height:6px;cursor:pointer;
}
.slider-row{margin-bottom:16px;}
.slider-row .slider-head{display:flex;justify-content:space-between;font-size:0.82rem;color:var(--text-mid);margin-bottom:6px;}
.slider-row .slider-head .val{color:var(--gold);font-family:'JetBrains Mono',monospace;}

.readout{
background:var(--cave-black);
border:1px solid var(--line);
border-radius:var(--radius-md);
padding:18px 20px;margin-top:18px;
}
.readout .big-num{font-family:'Baloo 2';font-size:2.1rem;color:var(--gold);line-height:1;}
.readout .big-num small{font-size:1rem;color:var(--text-dim);font-family:'Inter';font-weight:600;}
.readout .caption{font-size:0.9rem;color:var(--text-mid);margin-top:6px;}

.meter-track{height:16px;border-radius:100px;background:#0d0617;border:1px solid var(--line);overflow:hidden;margin-top:10px;}
.meter-fill{height:100%;border-radius:100px;background:linear-gradient(90deg,var(--crystal),var(--gold));width:0%;transition:width .4s ease;}
.meter-fill.danger{background:linear-gradient(90deg,var(--ember),var(--danger));}

button.btn{
font-family:'Inter';font-weight:700;font-size:0.92rem;
background:var(--ember);color:#fff8f0;border:none;
padding:12px 22px;border-radius:100px;cursor:pointer;
transition:transform .12s ease, background .2s ease;
}
button.btn:hover{background:var(--ember-bright);transform:translateY(-1px);}
button.btn.ghost{background:transparent;border:1px solid var(--line);color:var(--text-hi);}
button.btn.ghost:hover{border-color:var(--crystal);color:var(--crystal);}
.btn-row{display:flex;gap:12px;flex-wrap:wrap;margin-top:20px;}

.reveal-msg{
margin-top:18px;padding:16px 18px;border-radius:var(--radius-md);
background:rgba(55,214,196,0.08);border:1px solid rgba(55,214,196,0.35);
color:var(--text-hi);font-size:0.95rem;display:none;
}
.reveal-msg.show{display:block;}
.reveal-msg.bad{background:rgba(255,77,77,0.1);border-color:rgba(255,77,77,0.4);}

/* ----- kid definitions ----- */
.def-grid{display:grid;grid-template-columns:1fr 1fr;gap:22px;margin-top:24px;}
@media (max-width:700px){.def-grid{grid-template-columns:1fr;}}
.def-card{
border-radius:var(--radius-lg);padding:28px;
}
.def-card.entropy{background:linear-gradient(160deg,#2f1b4f,#1d1035);border:1px solid rgba(55,214,196,0.3);}
.def-card.crossent{background:linear-gradient(160deg,#3a1a12,#1d1035);border:1px solid rgba(232,84,30,0.35);}
.def-card h3{margin:0 0 10px;font-size:1.25rem;}
.def-card.entropy h3{color:var(--crystal);}
.def-card.crossent h3{color:var(--ember-bright);}
.def-card p{color:var(--text-mid);font-size:0.96rem;margin:0;}

```

```
footer{padding:40px 24px;text-align:center;color:var(--text-dim);font-size:0.85rem;}
```

```
.callout-strip{
display:flex;gap:14px;align-items:flex-start;
background:rgba(242,183,5,0.08);border:1px solid rgba(242,183,5,0.3);
border-radius:var(--radius-md);padding:16px 18px;margin-top:22px;
font-size:0.92rem;color:var(--text-mid);
}
```

```
.callout-strip .ic{font-size:1.3rem;flex-shrink:0;}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<nav class="topnav">
```

```
<div class="topnav-inner">
```

```
<span class="topnav-brand">  Dragon Egg Cave</span>
```

```
<a href="#story">The Cave</a>
```

```
<a href="#entropy">Confusion Game</a>
```

```
<a href="#crossentropy">Wrongness Game</a>
```

```
<a href="#recap">Recap</a>
```

```
</div>
```

```
</nav>
```

```
<header class="hero">
```

```
<div class="wrap">
```

```
<div class="hero-grid">
```

```
<div>
```

```
<span class="eyebrow-tag">A game to play!</span>
```

```
<h1 class="hero-title">The <em>Dragon Egg Cave</em></h1>
```

```
<p class="hero-sub">Play with a circle of dragon eggs and find out what "confusing" and "wrong" really mean — no reading required, just click, drag, and see what happens!</p>
```

```
<div class="badge-row">
```

```
<span class="badge">  Two games inside</span>
```

```
<span class="badge">  Click and drag to play</span>
```

```
</div>
```

```
</div>
```

```
<div>
```

```
<svg class="cave-scene" viewBox="0 0 400 320" xmlns="http://www.w3.org/2000/svg">
```

```
<path d="M0 320 L0 190 C 40 60, 140 10, 200 10 C 260 10, 360 60, 400 190 L400 320 Z" fill="#0d0617"/>
```

```
<path d="M20 320 L20 195 C 55 80, 145 30, 200 30 C 255 30, 345 80, 380 195 L380 320 Z" fill="#1d1035"/>
```

```
<g opacity="0.9">
```

```
<circle cx="110" cy="150" r="20" fill="#2f1b4f" stroke="#f6efe1" stroke-opacity="0.15"/>
```

```
<ellipse cx="110" cy="150" rx="13" ry="17" fill="#f6efe1" opacity="0.9"/>
```

```
<circle cx="200" cy="120" r="22" fill="#3a1a12" stroke="#f6efe1" stroke-opacity="0.15"/>
```

```
<ellipse cx="200" cy="120" rx="15" ry="20" fill="#f2b705"/>
```

```
<circle cx="290" cy="150" r="20" fill="#2f1b4f" stroke="#f6efe1" stroke-opacity="0.15"/>
```

```
<ellipse cx="290" cy="150" rx="13" ry="17" fill="#f6efe1" opacity="0.9"/>
```

```
</g>
```

```
<text x="200" y="126" font-size="22" text-anchor="middle">  </text>
```

```
<text x="110" y="156" font-size="18" text-anchor="middle">  </text>
```

```
<text x="290" y="156" font-size="18" text-anchor="middle">  </text>
```

```
<g>
```

```
<rect x="45" y="200" width="8" height="60" fill="#5c3a24"/>
```

```
<ellipse cx="49" cy="196" rx="12" ry="18" fill="#ff7a3d" opacity="0.85"/>
```



<div><strong>Try this:</strong> think of it like the old game <em>20 Questions</em> — every good yes/no question you ask can cut the eggs roughly in half. The more eggs there are, the more questions you'd need to find the dragon!</div>

</div>

</div>

<div class="panel" style="margin-top:24px;">

<p class="section-kicker" style="margin-bottom:4px;">// Level 2 — hints change everything</p>

<h3 style="margin:0 0 14px;color:var(--parchment);">What if some eggs feel warmer than others?</h3>

<p style="color:var(--text-mid);font-size:0.95rem;margin-top:0;">A real dragon egg gives off a little heat. Drag the sliders to say how "warm" each egg feels — the warmer it feels, the more likely it's the one. Watch what happens to the confusion meter as your hints get stronger.</p>

<div class="panel-grid">

<div>

<div class="egg-ring" id="ring-weighted"></div>

</div>

<div id="weighted-sliders"></div>

</div>

<div class="readout" style="margin-top:20px;">

<div class="big-num" id="entropy-weighted-val">2.00 <small>bits</small></div>

<div class="caption" id="entropy-weighted-caption">All four eggs feel equally warm — maximum confusion for 4 eggs.</div>

</div>

<div class="meter-track"><div class="meter-fill" id="meter-weighted"></div></div>

</div>

</div>

</section>

<section id="crossentropy">

<div class="wrap">

<p class="section-kicker">// Part 2</p>

<h2 class="section-title">Cross-Entropy — The Wrongness Meter</h2>

<div class="prose">

<p>Entropy measures confusion <em>before</em> anyone guesses. Cross-entropy measures something different: once a guess is made and the true egg is revealed, <strong>how wrong was that guess, really?</strong> Try both guessing styles below on the exact same hidden dragon egg.</p>

</div>

<div class="panel">

<div class="btn-row" style="margin-top:0;margin-bottom:6px;">

<button class="btn" id="new-round-btn"><img alt="refresh icon" data-bbox="325 658 340 670"/> Hide a new dragon egg</button>

</div>

<div class="panel-grid" style="margin-top:10px;">

<div>

<h3 style="margin:0 0 6px;color:var(--parchment);font-size:1.1rem;"><img alt="human icon" data-bbox="505 727 520 739"/> Human Guess</h3>

<p style="color:var(--text-dim);font-size:0.86rem;margin:0 0 14px;">Click exactly one egg. You're 100% sure — no doubts allowed.</p>

<div class="egg-ring" id="ring-human"></div>

</div>

<div>

<h3 style="margin:0 0 6px;color:var(--parchment);font-size:1.1rem;"><img alt="robot icon" data-bbox="505 812 520 824"/> Dragon-Bot Guess</h3>

<p style="color:var(--text-dim);font-size:0.86rem;margin:0 0 14px;">Drag belief points onto each egg. The bot is allowed to stay unsure.</p>

<div id="bot-sliders"></div>

</div>

</div>

<div class="btn-row">

<button class="btn" id="reveal-btn"><img alt="fire icon" data-bbox="295 928 310 940"/> Reveal the Dragon Egg!</button>

</div>

```

<div class="panel-grid" style="margin-top:10px;">
<div>
<div class="readout">
<div class="big-num" id="human-wrongness">— <small>bits</small></div>
<div class="caption" id="human-caption">Make a guess, then reveal.</div>
</div>
<div class="meter-track"><div class="meter-fill" id="meter-human"></div></div>
</div>
<div>
<div class="readout">
<div class="big-num" id="bot-wrongness">— <small>bits</small></div>
<div class="caption" id="bot-caption">Spread some belief, then reveal.</div>
</div>
<div class="meter-track"><div class="meter-fill" id="meter-bot"></div></div>
</div>
</div>
<div class="reveal-msg" id="compare-msg"></div>
</div>
</div>
</section>

```

```

<section id="recap">
<div class="wrap">
<p class="section-kicker">// Recap</p>
<h2 class="section-title">Kid-Sized Definitions</h2>
<div class="def-grid">
<div class="def-card entropy">
<h3>🌀 Entropy</h3>
<p><strong>How confusing the choices are</strong> — before anyone guesses. Lots of identical eggs = high entropy = very confusing. One egg glowing obviously = low entropy = barely any confusion at all.</p>
</div>
<div class="def-card crossent">
<h3>🌟 Cross-Entropy</h3>
<p><strong>How wrong your guess turned out to be</strong> — after the true egg is revealed. Being sure and right costs you nothing. Being sure and wrong costs you everything. Staying a little unsure is safer than it feels.</p>
</div>
</div>
</div>
</section>

```

```

<footer>🐉 The Dragon Egg Cave — a supplemental game about entropy & cross-entropy</footer>

```

```

<script>
(function(){
const TAU = Math.PI * 2;

function layout(n, radius){
const pts = [];
for(let i=0;i<n;i++){
const angle = -Math.PI/2 + (TAU * i)/n;
pts.push({x: 50 + radius*Math.cos(angle), y: 50 + radius*Math.sin(angle)});
}
return pts;
}
}

```

```

function makeRing(container, n, opts){
  opts = opts || {};
  container.innerHTML = "";
  const pts = layout(n, opts.radius || 38);
  const eggs = [];
  for(let i=0;i<n;i++){
    const p = pts[i];
    const div = document.createElement('div');
    div.className = 'egg';
    div.style.left = p.x + '%';
    div.style.top = p.y + '%';
    div.innerHTML = '<span class="egg-shape"> 🥚 </span>' + (opts.showPct ? '<span class="egg-pct" data-pct></span>' : "");
    container.appendChild(div);
    eggs.push(div);
  }
  return eggs;
}

```

```

/* ----- Section 1a: uniform entropy ----- */

```

```

const ringUniform = document.getElementById('ring-uniform');
const eggCountInput = document.getElementById('egg-count');
const eggCountVal = document.getElementById('egg-count-val');
const entropyUniformVal = document.getElementById('entropy-uniform-val');
const entropyUniformCaption = document.getElementById('entropy-uniform-caption');
const meterUniform = document.getElementById('meter-uniform');

```

```

function renderUniform(){
  const n = parseInt(eggCountInput.value, 10);
  eggCountVal.textContent = n;
  makeRing(ringUniform, n, {radius:38});
  const h = Math.log2(n);
  entropyUniformVal.innerHTML = h.toFixed(2) + ' <small>bits</small>';
  const q = Math.ceil(h);
  entropyUniformCaption.textContent = 'That\'s like needing about ' + q + ' good yes/no question' + (q===1?'':':s') + ' to find the dragon.';
  const pct = Math.min(100, (h / Math.log2(16)) * 100);
  meterUniform.style.width = pct + '%';
}
eggCountInput.addEventListener('input', renderUniform);
renderUniform();

```

```

/* ----- Section 1b: weighted entropy ----- */

```

```

const ringWeighted = document.getElementById('ring-weighted');
const weightedSlidersEl = document.getElementById('weighted-sliders');
const entropyWeightedVal = document.getElementById('entropy-weighted-val');
const entropyWeightedCaption = document.getElementById('entropy-weighted-caption');
const meterWeighted = document.getElementById('meter-weighted');
const WN = 4;
const weightedVals = [25,25,25,25];
let weightedEggs = [];

```

```

function buildWeightedSliders(){
  weightedSlidersEl.innerHTML = "";
  for(let i=0;i<WN;i++){
    const row = document.createElement('div');
    row.className = 'slider-row';

```

```

row.innerHTML = '<div class="slider-head"><span>Egg ' + (i+1) + ' warmth</span><span class="val" data-out-
'+i+'>'+weightedVals[i]+'</span></div>' +
'<input type="range" min="1" max="100" value="'+weightedVals[i]+' data-w-'+i+'>';
weightedSlidersEl.appendChild(row);
row.querySelector('input').addEventListener('input', (e)=>{
weightedVals[i] = parseInt(e.target.value,10);
row.querySelector("[data-out-'+i+'']").textContent = weightedVals[i];
renderWeighted();
});
}
}

```

```

function renderWeighted(){
weightedEggs = makeRing(ringWeighted, WN, {radius:38, showPct:true});
const sum = weightedVals.reduce((a,b)=>a+b,0);
const probs = weightedVals.map(v=>v/sum);
let h = 0;
probs.forEach(p=>{ if(p>0) h += -p*Math.log2(p); });
weightedEggs.forEach((egg,i)=>{
const pctEl = egg.querySelector("[data-pct]");
if(pctEl) pctEl.textContent = Math.round(probs[i]*100) + '%';
});
entropyWeightedVal.innerHTML = h.toFixed(2) + ' <small>bits</small>';
const maxH = Math.log2(WN);
const ratio = h / maxH;
let msg;
if(ratio > 0.9) msg = 'All four eggs feel about equally warm — maximum confusion for 4 eggs.';
else if(ratio > 0.5) msg = 'A couple of eggs feel warmer, but it\'s still pretty confusing.';
else if(ratio > 0.15) msg = 'One egg clearly feels warmest — not much confusion left.';
else msg = 'One egg is obviously the warmest — almost zero confusion!';
entropyWeightedCaption.textContent = msg;
meterWeighted.style.width = Math.min(100,(h/2)*100) + '%';
}
buildWeightedSliders();
renderWeighted();

```

```

/* ----- Section 2: cross-entropy game ----- */
const CN = 5;
const ringHuman = document.getElementById('ring-human');
const botSlidersEl = document.getElementById('bot-sliders');
const humanWrongnessEl = document.getElementById('human-wrongness');
const humanCaptionEl = document.getElementById('human-caption');
const botWrongnessEl = document.getElementById('bot-wrongness');
const botCaptionEl = document.getElementById('bot-caption');
const meterHuman = document.getElementById('meter-human');
const meterBot = document.getElementById('meter-bot');
const compareMsg = document.getElementById('compare-msg');
const revealBtn = document.getElementById('reveal-btn');
const newRoundBtn = document.getElementById('new-round-btn');

```

```

let trueIndex = Math.floor(Math.random()*CN);
let humanGuess = null;
let botVals = new Array(CN).fill(20);
let humanEggs = [];

```

```

function buildHumanRing(){

```

```

humanEggs = makeRing(ringHuman, CN, {radius:38});
humanEggs.forEach((egg,i)=>{
egg.classList.remove('selected','true-egg');
egg.querySelector('.egg-shape').textContent = '🥚';
egg.onclick = () => {
humanGuess = i;
humanEggs.forEach(e=>e.classList.remove('selected'));
egg.classList.add('selected');
};
});
}

function buildBotSliders(){
botSlidersEl.innerHTML = "";
for(let i=0;i<CN;i++){
const row = document.createElement('div');
row.className = 'slider-row';
row.innerHTML = '<div class="slider-head"><span>Egg ' + (i+1) + ' belief</span><span class="val" data-bout-' + i + '>' + botVals[i] + '</span></div>' +
'<input type="range" min="0" max="100" value="' + botVals[i] + '" data-b-' + i + '>';
botSlidersEl.appendChild(row);
row.querySelector('input').addEventListener('input', (e)=>{
botVals[i] = parseInt(e.target.value,10);
row.querySelector('[data-bout-' + i + ']').textContent = botVals[i];
});
}
}

function resetRound(){
trueIndex = Math.floor(Math.random()*CN);
humanGuess = null;
botVals = new Array(CN).fill(20);
buildHumanRing();
buildBotSliders();
humanWrongnessEl.innerHTML = '— <small>bits</small>';
humanCaptionEl.textContent = 'Make a guess, then reveal.';
botWrongnessEl.innerHTML = '— <small>bits</small>';
botCaptionEl.textContent = 'Spread some belief, then reveal.';
meterHuman.style.width = '0%';
meterHuman.classList.remove('danger');
meterBot.style.width = '0%';
meterBot.classList.remove('danger');
compareMsg.classList.remove('show','bad');
}

function reveal(){
humanEggs.forEach((egg,i)=>{
if(i===trueIndex){ egg.classList.add('true-egg'); egg.querySelector('.egg-shape').textContent = '🥚'; }
});

// Human
if(humanGuess === null){
humanCaptionEl.textContent = 'Pick an egg first!';
} else if(humanGuess === trueIndex){
humanWrongnessEl.innerHTML = '0.00 <small>bits</small>';

```

```

humanCaptionEl.textContent = 'Dead right — zero wrongness!';
meterHuman.style.width = '0%';
meterHuman.classList.remove('danger');
} else {
humanWrongnessEl.innerHTML = '∞ <small>bits</small>';
humanCaptionEl.textContent = 'You were 100% sure — and 100% wrong. The wrongness meter goes off the charts.';
meterHuman.style.width = '100%';
meterHuman.classList.add('danger');
}

// Bot
const sum = botVals.reduce((a,b)=>a+b,0) || 1;
const probs = botVals.map(v=>v/sum);
const qTrue = probs[trueIndex];
let botLoss;
if(qTrue <= 0.0001){
botLoss = 12; // display cap
botWrongnessEl.innerHTML = '12+ <small>bits</small>';
} else {
botLoss = -Math.log2(qTrue);
botWrongnessEl.innerHTML = botLoss.toFixed(2) + ' <small>bits</small>';
}
botCaptionEl.textContent = 'The bot put ' + Math.round(qTrue*100) + '% belief on the true egg.';
meterBot.style.width = Math.min(100,(botLoss/6)*100) + '%';
if(botLoss > 3) meterBot.classList.add('danger'); else meterBot.classList.remove('danger');

if(humanGuess !== null){
compareMsg.classList.add('show');
if(humanGuess !== trueIndex && qTrue > 0.15){
compareMsg.classList.remove('bad');
compareMsg.innerHTML = '<strong>Notice:</strong> the human guess was wrong and total (infinite wrongness), but the Dragon-Bot spread some belief onto the true egg anyway — so it only got a mild penalty, and it now has useful information for its next guess.';
} else if(humanGuess === trueIndex){
compareMsg.classList.remove('bad');
compareMsg.innerHTML = '<strong>Notice:</strong> a lucky certain guess costs nothing — but it also teaches the bot nothing about staying safely unsure next time.';
} else {
compareMsg.classList.add('bad');
compareMsg.innerHTML = '<strong>Notice:</strong> both guesses missed the true egg badly this round. A confident wrong guess — human or bot — is always the most expensive kind of mistake.';
}
}
}

revealBtn.addEventListener('click', reveal);
newRoundBtn.addEventListener('click', resetRound);
resetRound();
})();
</script>
</body>
</html>

```